

CSA0613 - Design and Analysis of Algorithms

CO3 AT1 Experiments-Based Assessment

Divyesh S P (192424147)

Evaluation of Quick Sort Efficiency on Large-Scale Distributed Datasets by Simulating Partitioning Across Nodes

Aim

To evaluate the efficiency of the Quick Sort algorithm on a large-scale distributed dataset by simulating data partitioning across multiple nodes, measuring execution time and communication overhead, and analyzing the impact of distributed processing on performance.

Algorithm

1. Generate a large dataset containing 100,000 random integers.
2. Divide the dataset equally into four partitions (nodes).
3. Apply the Quick Sort algorithm independently on each partition.
4. Measure the total sorting time.
5. Merge all sorted partitions into a single sorted dataset.
6. Measure the merge time.
7. Calculate the communication overhead between nodes.
8. Compute the total execution time.
9. Display the sorted output and performance metrics.

Implementation

- **Programming Language:** Python
- **Dataset Size:** 100,000 integers
- **Number of Nodes:** 4
- **Sorting Algorithm:** Quick Sort
- **Merge Technique:** Heap Merge (heapq.merge)

Python Code

```
import random
import time
import heapq

# -----
# Quick Sort Function
# -----

def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr)//2]

    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)

# -----
# Dataset Generation
# -----

SIZE = 100000

dataset = [random.randint(1,1000000) for _ in range(SIZE)]

# -----
# Simulated Nodes
# -----

nodes = 4

chunk_size = len(dataset)//nodes

partitions = []

for i in range(nodes):
    start = i * chunk_size
```

```

        if i == nodes-1:
            end = len(dataset)
        else:
            end = (i+1) * chunk_size

        partitions.append(dataset[start:end])

# -----
# Sorting
# -----

sorting_start = time.time()

sorted_partitions = []

for part in partitions:
    sorted_partitions.append(quick_sort(part))

sorting_end = time.time()

sorting_time = sorting_end - sorting_start

# -----
# Merge
# -----

merge_start = time.time()

merged = list(heapq.merge(*sorted_partitions))

merge_end = time.time()

merge_time = merge_end - merge_start

# -----
# Communication Overhead
# -----

communication_overhead = nodes * 0.01

total_time = sorting_time + merge_time + communication_overhead

# -----
# Output
# -----

print("Dataset Size :", SIZE)
print("Nodes :", nodes)
print("Sorting Time :", round(sorting_time,4),"seconds")
print("Merge Time :", round(merge_time,4),"seconds")
print("Communication Overhead :", communication_overhead,"seconds")
print("Total Execution Time :", round(total_time,4),"seconds")

print("\nFirst 20 Sorted Values")

print(merged[:20])

```

Output

```
... Dataset Size : 100000
    Nodes : 4
    Sorting Time : 0.284 seconds
    Merge Time : 0.0384 seconds
    Communication Overhead : 0.04 seconds
    Total Execution Time : 0.3624 seconds

First 20 Sorted Values
[7, 7, 44, 47, 50, 53, 69, 71, 76, 80, 81, 99, 105, 108, 108, 110, 124, 131, 137, 140]
```

Performance Analysis

Parameter	Value
Dataset Size	100,000
Number of Nodes	4
Sorting Time	0.284 s
Merge Time	0.0384 s
Communication Overhead	0.04 s
Total Execution Time	0.3624 s

Interpretation

- The dataset of **100,000 elements** was successfully divided into **4 simulated nodes**.
- Each node independently sorted its assigned partition using the Quick Sort algorithm.
- The total sorting process took **0.284 seconds**, indicating that Quick Sort performs efficiently on large datasets.
- The merge operation required only **0.0384 seconds**, showing that combining sorted partitions introduced minimal delay.

- The simulated communication overhead was **0.04 seconds**, representing the time required for data exchange between distributed nodes.
- The overall execution time of **0.3624 seconds** demonstrates that partitioning data across multiple nodes can efficiently process large datasets while keeping communication costs relatively low.

Advantages

- Efficient average-case time complexity of **$O(n \log n)$** .
- Supports parallel processing by sorting partitions independently.
- Reduces the workload on individual nodes.
- Suitable for handling large-scale datasets.
- Produces accurate and correctly sorted output.

Limitations

- Communication overhead increases as the number of nodes increases.
- Additional time is required to merge sorted partitions.
- Uneven data distribution among nodes may reduce overall performance.
- Worst-case time complexity is **$O(n^2)$** when poor pivot selection occurs.

Result

The Quick Sort algorithm was successfully implemented on a simulated distributed environment using four nodes. The program efficiently sorted a dataset of **100,000 elements** with a sorting time of **0.284 seconds**, merge time of **0.0384 seconds**, communication overhead of **0.04 seconds**, and a total execution time of **0.3624 seconds**. The results indicate that Quick Sort is well suited for distributed systems, provided that communication overhead is minimized and data is evenly partitioned across nodes.